

Project Title

SE – 4111 Course Project

Submitted to: Dr. Mawal A. Mohammed

Prepared by:

S	ID	Name
1		
2		
3		
4		

Nov. 2026

Table of Contents

1. **Requirements**.....4
2. **Design**.....4
3. **Implementation**.....6
4. **Lessons learned**.....7

Important Notes:

1. **Font**: Times new romans
2. **Section titles** (size: 14; Bold)
3. **Section text** (size:12; space between lines 1.5; justification: fill justification)

Executive Summary

In this section, you will write the summary of your project. You can leave this section until the end of the project.

1. Requirements

This section is dedicated to introducing your project, describing its objectives and features. The list of 6 requirements should be presented in this section.

2. Design

In this section, technical and design decisions should be presented here as in the example below: (Replace red Text in the table with your design and technical decisions)

Example Integration: To-Do List Application

Technical and design requirements	Description of integration into your application
Architectural Pattern	The application will be structured using **MVC**: • **Model:** Consists of the `Task` class. It manages all the to-do items and contains the logic for adding, deleting, editing, and marking tasks as complete. It is completely independent of the user interface. • **View:** The main window of the application. It visually displays the list of tasks, perhaps with checkboxes and text labels. It observes the model for any changes. • **Controller:** Contains the event handlers for GUI elements. It listens for user actions like clicking the "Add Task" button or checking a box, and it calls the appropriate methods on the Model to update the application's state.
Design Patterns	The **Observer** pattern will be implemented. The `Task` (Model) will be the "Subject." The GUI component that displays the list of tasks (View) will be the "Observer." **Justification:** When a task is added, removed, or modified in the `Task`, the `Task` will notify its observers. The View will then automatically refresh itself to reflect changes. This decouples the Model from the View.
Abstract Data Types	In addition to the required classes to get the app working, the

Technical and design requirements	Description of integration into your application

(ADTs)	primary ADT for the model will be:
	
	`Task`: (immutable) Represents a single to-do item. Its abstract value includes a description, a completion status, and a unique ID
	//RI:
	//id > 0 (unique identifier must be a positive integer)
	//id is unique across all Task instances (no two tasks //share the same id)
	//description != null
	//description.length() > 0 (description must not be //empty)
	
	//AF
	// Represents a single item in a to-do list, uniquely //identified by 'id', described by the text in //'description', and considered finished if //'isCompleted' is true, otherwise it is pending.

Equality	The `equals()` and `hashCode()` methods will be implemented for the **`Task` ADT**. **Justification:** This is essential for the design. It allows `Task` objects to be compared and used reliability in a `HashMap`, which is an efficient way for the `Task` model to store and look up specific tasks.

3. Implementation

In this section, you will provide the screenshots of the application. You will also provide a link to the code in GitHub.

4. Lessons learned.

The lessons you learned from conducting this project should be presented here.